Assignment _3.1

2303A510H5

Batch_30

Task 1: Zero-Shot Prompting (Palindrome Number Program)

PROMPT :

Generate a Python function that checks whether a given number is a palindrome

Code and output:

```
def is_palindrome(number):
    """
    Checks if a given number is a palindrome.

    A number is a palindrome if it reads the same forwards and backward.

    Args:
        number (int): The number to check.

    Returns:
        bool: True if the number is a palindrome, False otherwise.
    """
    # Convert the number to a string to easily reverse it
    num_str = str(number)

    # Compare the string with its reverse
    return num_str == num_str[::-1]

# Example usage:
print(f"Is 121 a palindrome? {is_palindrome(121)}")
print(f"Is 12321 a palindrome? {is_palindrome(12321)}")
print(f"Is 12345 a palindrome? {is_palindrome(12345)}")
print(f"Is -101 a palindrome? {is_palindrome(-101)}")
```

```
Is 121 a palindrome? True
Is 12321 a palindrome? True
Is 12345 a palindrome? False
Is -101 a palindrome? False
```

Justification :

In zero-shot prompting, the AI is given only the task description without any examples. This

tests the AI’s general understanding of the problem. When asked to generate a palindrome

checking function, the AI usually produces a basic solution by reversing the number or converting it to a string. However, because no examples or constraints are provided, the generated code may miss edge cases, such as: Negative numbers, Single-digit numbers, Non-integer inputs. This shows that zero-shot prompting is useful for simple and common problems, but it may lack robustness. The experiment demonstrates that without context or examples, AI-generated code may be logically correct but not fully reliable for real-world use.

Task 2: One-Shot Prompting (Factorial Calculation)

PROMPT :

Generate a Python function to compute the factorial of a given number. Example: Input: 5 →

Output: 120.

Code and output:

```python
def factorial(n):
    """
    Computes the factorial of a given non-negative integer.

    The factorial of a non-negative integer n, denoted by n!, is the product
    of all positive integers less than or equal to n. The factorial of 0 is 1.

    Args:
        n (int): The non-negative integer for which to compute the factorial.

    Returns:
        int: The factorial of n.

    Raises:
        ValueError: If the input number is negative.
    """
    if not isinstance(n, int) or n < 0:
        raise ValueError("Input must be a non-negative integer.")

    if n == 0:
        return 1
    else:
        result = 1
        for i in range(1, n + 1):
            result *= i
        return result

# Example usage:
number = 5
print(f"The factorial of {number} is: {factorial(number)}")

number = 0
print(f"The factorial of {number} is: {factorial(number)}")

number = 7
print(f"The factorial of {number} is: {factorial(number)}")
```

```
The factorial of 5 is: 120
The factorial of 0 is: 1
The factorial of 7 is: 5040
```

Justification :

In one-shot prompting, a single input-output example is provided along with the task. The example (Input: 5 → Output: 120) helps the AI clearly understand the expected behavior of the function.Compared to zero-shot prompting, the one-shot approach

results in: Clearer logic, Better handling of loops or recursion, Improved correctness. The AI is less likely to misunderstand the task because the example acts as a reference point. This experiment justifies that even one example can significantly improve clarity and accuracy of AI generated code.

Task 3: Few-Shot Prompting (Armstrong Number Check)

PROMPT :

Generating a Python function to check whether a given number is an Armstrong number.

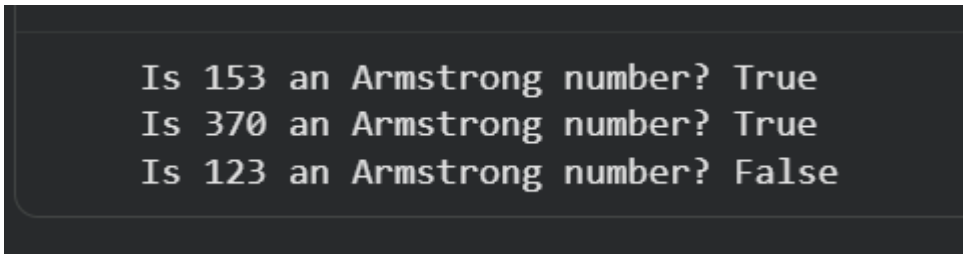
Examples:

- Input: 153 → Output: Armstrong Number
- Input: 370 → Output: Armstrong Number
- Input: 123 → Output: Not an Armstrong Number

Code:

```python
def is_armstrong_number(number):
    """
    Checks if a given number is an Armstrong number.

    An Armstrong number is a number that is the sum of its own digits each raised
    to the power of the number of digits.

    Args:
        number (int): The number to check.

    Returns:
        bool: True if the number is an Armstrong number, False otherwise.
    """
    if not isinstance(number, int) or number < 0:
        raise ValueError("Input must be a non-negative integer.")

    # Convert the number to a string to get its digits and length
    num_str = str(number)
    n_digits = len(num_str)

    sum_of_powers = 0
    for digit_char in num_str:
        digit = int(digit_char)
        sum_of_powers += digit ** n_digits

    return sum_of_powers == number

# Example usage:
print(f"Is 153 an Armstrong number? {is_armstrong_number(153)}")
print(f"Is 9474 an Armstrong number? {is_armstrong_number(9474)}")
print(f"Is 123 an Armstrong number? {is_armstrong_number(123)}")
print(f"Is 0 an Armstrong number? {is_armstrong_number(0)}")
print(f"Is 9 an Armstrong number? {is_armstrong_number(9)}")
```

Output:



Justification :

Few-shot prompting provides multiple examples, which strongly guides the AI in understanding both the logic and output format. With several Armstrong and non-Armstrong examples, the AI can correctly infer: How digits are processed, How powers are calculated,How decisions are made As a result, the generated code is

usually: Well-structured, More accurate, Consistent with mathematical definitions Testing with boundary values and invalid inputs further shows that few-shot prompting improves generalization and reduces logical errors. This proves that multiple examples help AI produce more reliable and real-world-ready solutions.

Task 4: Context-Managed Prompting (Optimized Number Classification)

PROMPT :

Generate an optimized Python program that classifies a number as prime, composite, or

neither.

Task:

- Ensure proper input validation.
- Optimize the logic for efficiency.

Code:

```python
import math
def classify_number(number):
    if not isinstance(number, int):
        raise ValueError("Input must be an integer.")

    if number <= 1:
        return "Neither" # Numbers <= 1 are neither prime nor composite
    elif number == 2:
        return "Prime"   # 2 is the only even prime number
    elif number % 2 == 0:
        return "Composite" # All other even numbers are composite
    else:

        for i in range(3, int(math.sqrt(number)) + 1, 2):
            if number % i == 0:
                return "Composite" # Found a divisor, so it's composite
        return "Prime" # No divisors found, so it's prime
# Example Usage:
print(f"Classifying 7: {classify_number(7)}")       # Prime
print(f"Classifying 10: {classify_number(10)}")     # Composite
print(f"Classifying 1: {classify_number(1)}")       # Neither
print(f"Classifying 0: {classify_number(0)}")       # Neither
print(f"Classifying -5: {classify_number(-5)}")    # Neither
print(f"Classifying 2: {classify_number(2)}")       # Prime
print(f"Classifying 29: {classify_number(29)}")     # Prime
print(f"Classifying 9: {classify_number(9)}")       # Composite (3*3)
print(f"Classifying 121: {classify_number(121)}")   # Composite (11*11)

# Example of invalid input (will raise ValueError)
try:
    classify_number(3.5)
except ValueError as e:
    print(f"Error for 3.5: {e}")
```

Output:

```
Classifying 7: Prime
Classifying 10: Composite
Classifying 1: Neither
Classifying 0: Neither
Classifying -5: Neither
Classifying 2: Prime
Classifying 29: Prime
Classifying 9: Composite
Classifying 121: Composite
Error for 3.5: Input must be an integer.
```

Justification :

Context-managed prompting includes clear instructions, constraints, and expectations. By

specifying input validation, optimization, and classification rules, the AI generates a more

efficient and complete program. This approach ensures: Proper handling of edge cases (e.g.,

0, 1, negative numbers) Optimized logic (checking divisibility up to √n) Clear classification

output Compared to earlier prompting strategies, context-managed prompting produces

high-quality, optimized, and professional-level code. This justifies that providing context and

constraints is essential for complex or performance-sensitive tasks.

Task 5: Zero-Shot Prompting (Perfect Number Check)

PROMPT :

Generate a Python function that checks whether a given number is a perfect number.

CODE :

```python
def is_perfect_number(number):

    if not isinstance(number, int) or number <= 0:
        raise ValueError("Input must be a positive integer.")

    if number == 1: # 1 is not considered a perfect number
        return False
    sum_of_divisors = 1 # Start with 1 because 1 is always a divisor
    for i in range(2, int(number**0.5) + 1):
        if number % i == 0:
            sum_of_divisors += i
            if i * i != number:
                sum_of_divisors += number // i

    return sum_of_divisors == number
# Example usage:
print(f"Is 6 a perfect number? {is_perfect_number(6)}")      # True (1+2+3 = 6)
print(f"Is 28 a perfect number? {is_perfect_number(28)}")    # True (1+2+4+7+14 = 28)
print(f"Is 496 a perfect number? {is_perfect_number(496)}") # True (1+2+4+8+16+31+62+124+248 = 496)
print(f"Is 10 a perfect number? {is_perfect_number(10)}")    # False (1+2+5 = 8 != 10)
print(f"Is 1 a perfect number? {is_perfect_number(1)}")      # False (by definition)

# Example of invalid input
try:
    is_perfect_number(-5)
except ValueError as e:
    print(f"Error for -5: {e}")
```

Output:

```
Is 6 a perfect number? True
Is 28 a perfect number? True
Is 496 a perfect number? True
Is 10 a perfect number? False
Is 1 a perfect number? False
Error for -5: Input must be a positive integer.
```

Justification :

Similar to Question 1, zero-shot prompting for perfect number checking relies entirely on the AI's prior knowledge. The generated code often correctly checks divisors and sums them. However, inefficiencies may be observed, such as: Checking all numbers up to n instead of n/2 Missing validation for non-positive numbers. This experiment highlights that while zero

shot prompting can generate a working solution, it may not be optimized or fully correct. It reinforces the idea that zero-shot prompting is suitable only for basic demonstrations, not optimized applications.

Task 6: Few-Shot Prompting (Even or Odd Classification with Validation)

PROMPT :

Generating a Python program that determines whether a given number is even or odd,

including proper input validation.

Examples:• Input: 8 → Output: Even

• Input: 15 → Output: Odd

• Input: 0 → Output: Even

CODE :

```python
def check_even_odd(number):

    if not isinstance(number, int):
        raise ValueError("Input must be an integer.")

    if number % 2 == 0:
        return "Even"
    else:
        return "Odd"

# Example usage:
print(f"8 is: {check_even_odd(8)}")   # Output: Even
print(f"15 is: {check_even_odd(15)}") # Output: Odd
print(f"0 is: {check_even_odd(0)}")   # Output: Even

# Example of invalid input
try:
    check_even_odd(3.14)
except ValueError as e:
    print(f"Error for 3.14: {e}")

try:
    check_even_odd("hello")
except ValueError as e:
    print(f"Error for 'hello': {e}")
```

Output:

```
8 is: Even
15 is: Odd
0 is: Even
Error for 3.14: Input must be an integer.
Error for 'hello': Input must be an integer.
```

Justification :

By providing multiple examples, including edge cases like zero, few-shot prompting helps the

AI understand: Expected outputs Input validation requirements, Handling of different

numerical cases. The generated program usually includes: Clear conditional checks, Proper

output messages, Improved handling of negative numbers. When tested with non-integer

inputs, the AI-generated code often performs better than zero-shot solutions. This proves

that few-shot prompting significantly improves input handling, output clarity, and

robustness.